

ResumeIQ – AI-Based Resume Screening and Job Matching System

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

SADHANA SHANMUGAM - 2104251040837

DHANYA SRI MOHANDASS - 2104251040189

Submitted in partial fulfilment of the requirements

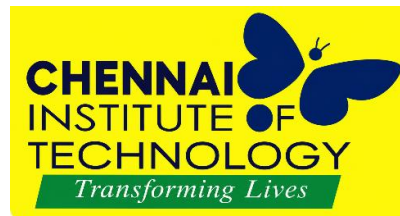
for the

Project-Based Learning component of Machine Learning

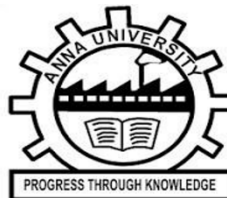
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai

(Autonomous)

OCTOBER 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To create next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contribute towards the research ecosystem by providing a suitable,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled **“ResumeIQ – AI-Based Resume Screening and Job Matching System”** is a Bonafide record of work carried out by **Sadhana Shanmugam [2104251040837]** , **Dhanya Sri Mohandass [2104251040189]** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year [2026–2027] under my supervision.

SIGNATURE

Dr. S. PAVITHRA, M.E., Ph.D.,
Professor and Head,
Dept. of Computer Science and Engineering
Chennai Institute of Technology,
Chennai – 69.

SIGNATURE

Ms. SWATHI L,
MENTOR
Assistant Professor
Dept. of Computer Science and Engineering
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on

Internal Examiner

External Examiner

DECLARATION

I/We jointly declare that the PBL report on “**ResumeIQ – AI-Based Resume Screening and Job Matching System**” is the result of original work done by us and best of our knowledge, similar work has not been submitted to “**ANNA UNIVERSITY, CHENNAI**” for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.



SIRAGU

Signature

Sadhana Shanmugam

Dhanya Sri Mohandass

Place: Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proud to render our thanks to our Principal **Dr. A. RAMESH M.E., Ph.D.**, for the facilities and the encouragement given by him toward the progress and completion of our project.

We would like to express special thanks and gratitude to our Dean **Dr. V. SRINIVASA RAO M.E., Ph.D.**, who has been a key source of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E., Ph.D.**, for her effective leadership, encouragement and guidance throughout the project.

We would like to extend our thanks to the Project Co-ordinator **Ms. SWATHI L**, Assistant Professor, Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from our class advisors **Dr. G. IRIN LORETTA M.E.**, Assistant Professor, and **S.E. NEELA KANDAN M.TECH.**, Assistant Professor, of the Department of Computer Science and Engineering for their valuable suggestions and support toward the successful completion of the project.

Sadhana Shanmugam (2104251040837)

Dhanya Sri Mohandass (2104251040189)

ABSTRACT

Recruiters routinely receive hundreds of resumes for a single vacancy, and screening them by hand is slow, inconsistent and prone to human error and bias, so qualified candidates can be overlooked. ResumeIQ is an AI-based resume analysis and job-matching system that automates this first screening stage. Resumes in PDF, DOC, DOCX and TXT formats are converted to text, and a rule-based natural-language-processing layer extracts skills (from a 164-skill taxonomy), education level, years of experience and contact details. Names, e-mail addresses and phone numbers are removed before scoring so that they cannot influence a ranking. The cleaned text is converted into TF-IDF vectors (unigrams and bigrams, 11,688 features), classified into one of 12 job roles by a Multinomial Naive Bayes model and a calibrated linear Support Vector Machine whose probabilities are averaged, and compared with a job description by cosine similarity. A transparent five-part match score (text similarity 40%, skill coverage 30%, role fit 15%, experience 10% and education 5%) ranks the candidates and produces a top-N shortlist with matched, related and missing skills for every candidate; the same engine also recommends the best-fit roles for a single resume. The system is delivered as a Flask web application with a JSON API and is packaged with Docker and a Render blueprint. The models were trained on 2,400 synthetic resumes (12 roles $\times$ 200) because real resumes are personal data. On a held-out test set of 480 resumes the ensemble reached 99.8% accuracy (479/480), with 5-fold cross-validation accuracy of 99.7% (Naive Bayes) and 99.4% (SVM); all five hand-written resumes that the data generator never produced were matched to the correct role, and 14 sample resumes in four file formats were screened in a median of 0.19 s. Because the training data are synthetic, these figures show that the pipeline works and are not an estimate of real-world accuracy; the report states this limitation openly and documents a code-ready path to retrain on real labelled resumes.

Keywords: Resume Screening, Job Matching, Natural Language Processing, TF-IDF, Naive Bayes, Support Vector Machine.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Driving Question	2
1.3 Objectives	3
1.4 Scope and Limitations	3
CHAPTER 2: CONCEPT EXPLORATION	4
2.1 Related Approaches	4
2.2 Summary Table	5
2.3 What This Told Us	6
CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION	6
3.1 Weekly PBL Progress Log	6
3.2 Requirements	7
3.3 Feasibility	8
CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT	8
4.1 System Architecture	8
4.2 Iteration 1: Baseline	9
4.3 Iteration 2: Refinement	10
4.4 Final Approach	11
4.5 Training Procedure	13
CHAPTER 5: IMPLEMENTATION	14
5.1 Module Description	14
5.2 Key Code Snippets	15
5.3 User Interface / Demo	17
5.4 Interfaces and Deployment	19
CHAPTER 6: RESULTS AND DISCUSSION	20
6.1 Evaluation Metrics	20
6.2 Results Across Iterations	21
6.3 Discussion	23
6.4 Limitations	24
CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES	25
7.1 Individual Reflections	25
7.2 Team Learning	25
7.3 Course Outcomes — Evidence Summary	26
CHAPTER 8: CONCLUSION AND FUTURE SCOPE	27
8.1 Conclusion	27
8.2 Future Scope	27
REFERENCES	28
APPENDIX	29
A.1 Full Source Code	29
A.2 Complete Weekly PBL Log	29
A.3 Self and Peer Assessment	29

LIST OF FIGURES

Figure 4.1 System Architecture Diagram	9
Figure 4.2 ML Scoring Pipeline Data Flow	12
Figure 5.1 Home Page of ResumeIQ	17
Figure 5.2 Screen Candidates Page with a Job Description and Resumes	17
Figure 5.3 Screening Results: Ranked Candidates with Skill Evidence	18
Figure 5.4 Find Job Match: Role Recommendations for One Resume	18
Figure 5.5 Model Insights Page	19
Figure 6.1 Evaluation Metrics: Final Model and Accuracy Across Data Iterations	21
Figure 6.2 Confusion Matrix of the Ensemble on the Hold-out Set	22
Figure 6.3 Live Ranking of the Sample Resumes for the Data Scientist Job	23
Figure 7.1 Course Outcomes	26



LIST OF TABLES

Table 2.1 Summary of Related Approaches	5
Table 3.1 Weekly PBL Progress Log	7
Table 3.2 Hardware and Software Requirements	8
Table 4.1 Components of the Match Score	12
Table 4.2 Job Roles Modelled and Their Core Skills	13
Table 4.3 Dataset and Model Configuration	14
Table 5.1 Web Pages and JSON API Endpoints	19
Table 5.2 Automated Test Suite (34 tests)	20
Table 6.1 Model Evaluation Results Across Iterations	22
Table 6.2 Per-role Results of the Ensemble on the Hold-out Set	23
Table 6.3 Top Candidates for the Data Scientist Job (14 Sample Resumes)	23



LIST OF ABBREVIATIONS

Abbreviation	Full Form
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency – Inverse Document Frequency
NB	Naive Bayes
SVM	Support Vector Machine
CV	Cross-Validation
PBL	Project-Based Learning
PII	Personally Identifiable Information
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
PDF	Portable Document Format
DOCX	Office Open XML Word Document
OCR	Optical Character Recognition
NER	Named Entity Recognition
CSP	Content Security Policy
CI	Continuous Integration
UI	User Interface
WSGI	Web Server Gateway Interface

TEAM ROLES AND RESPONSIBILITIES

Both members jointly owned problem framing, the weekly mentor-review cycle, the literature exploration in Chapter 2, and the final report, and contributed **equally (50% : 50%)** to the project. Division of the build work below reflects primary ownership; both members reviewed and tested the full system before each milestone.

Team Member	Primary Role	Key Responsibilities
Sadhana Shanmugam (2104251040837)	ML Pipeline & Backend Lead	Designed and implemented the resume text extraction (extractor.py), the rule-based NLP parser and skill taxonomy (parser.py, skills.py), the TF-IDF / Naive Bayes / SVM training and evaluation pipeline (train.py, dataset.py), the scoring and role-recommendation engine (engine.py), and the Flask JSON API.
Dhanya Sri Mohandass (2104251040189)	Interface, Testing & Deployment Lead	Designed and implemented the web interface (templates, CSS and JavaScript for the screening, job-match and model-insight pages), prepared the sample resumes and screenshots, wrote the automated test suite, containerised the application with Docker and prepared the Render blueprint and CI workflow.

CHAPTER 1 INTRODUCTION

1.1 Background

Hiring begins with a screening problem. A single advertised vacancy can attract hundreds or thousands of applications, and recruiters must decide, often in minutes per resume, which of them deserve an interview. Done by hand, this first-pass screening is slow, tiring and inconsistent: two reviewers may rank the same resume differently, one reviewer may judge differently at the start and the end of a long day, and a qualified candidate can be overlooked simply because the resume uses different wording from the job advertisement. Applicant tracking systems that filter on exact keywords reduce the workload but inherit the same weakness, because “machine learning” and “ML”, or “scikit-learn” and “sklearn”, are different strings to a keyword filter but the same skill to a person.

Resumes also arrive as unstructured documents in several file formats (PDF, Word and plain text) with no common layout. Before any comparison can be made, the text must be extracted, the sections must be located, and facts such as skills, education and years of experience must be recovered from free text. Machine learning offers a natural way to handle the comparison step: a text-classification model can learn which vocabulary characterises each job family, and a similarity measure over weighted term vectors can quantify how closely a resume matches a particular job description.

Automation is not automatically fair. A review of algorithmic-hiring practice found that vendors say little about how bias is actually validated [1], and a widely reported case in 2018 involved a company withdrawing an experimental machine-learning recruiting tool after it was found to disadvantage women [2]. A screening system should therefore be explainable, should keep personal identifiers out of its scoring, and should be positioned as a decision aid rather than a decision maker.

ResumeIQ was undertaken as the Project-Based Learning (PBL) component of the Machine Learning course (CS5305). It was deliberately scoped around the build–learn cycle that PBL is meant to teach: a baseline text classifier, a refinement driven by what the first results revealed, and a final system that the team can explain component by component rather than point at as a black box.

1.2 Driving Question

Can a classical machine-learning pipeline built from TF-IDF features, Naive Bayes and Support Vector Machine classifiers, and cosine similarity turn unstructured resumes into a ranked, explainable shortlist for a given job description, and recommend suitable job roles for a single resume, without using the candidate's name or contact details and while running on ordinary laptop hardware? And if two structurally different classifiers are used, does combining them give a more reliable role signal than either one alone?

This question narrows into a concrete, buildable task: extract skills, education and experience from resumes with NLP; represent resume and job text as TF-IDF vectors; classify each resume into one of twelve job roles with Naive Bayes and an SVM; and combine text similarity, skill coverage, role agreement, experience and education into a single score that a recruiter can inspect rather than simply trust.

1.3 Objectives

- To accept resumes in PDF and DOC formats (and, in addition, DOCX and plain text) and convert them reliably into text.
- To extract key details (skills, education and experience) using natural-language-processing techniques, and to remove names and contact details before scoring.
- To match resumes with job descriptions and measure suitability using TF-IDF vectors and cosine similarity.
- To apply TF-IDF, Naive Bayes and SVM for ranking and classification, and to evaluate them with a held-out test set and cross-validation.
- To generate a shortlist of the top candidates automatically, with an explanation (matched, related and missing skills) for every candidate, and to recommend suitable job roles for a single resume.
- To deliver the system as a working website built with Python, Scikit-learn and Pandas, packaged for deployment on Render, and to document the weekly PBL progress and what the process taught each member.

1.4 Scope and Limitations

ResumeIQ models 12 job roles (Data Scientist, Machine Learning Engineer, Data Analyst, Data Engineer, Backend Developer, Frontend Developer, Full Stack Developer, DevOps Engineer, Mobile App Developer, QA Engineer, Cybersecurity Analyst and Business Analyst) and works on English-language, text-based resumes. Skills are recognised from a curated taxonomy of 164 skills with aliases (for example “sklearn” for Scikit-learn), so skills outside the taxonomy are not detected. Scanned or image-only documents contain no text layer and are rejected with a clear message; optical character recognition is outside the scope of this build.

The classifiers were trained on 2,400 synthetic resumes generated from the role catalog, because real resumes are personal data and no real labelled corpus was available to the team. The reported accuracy therefore measures how well the pipeline separates the synthetic roles and must not be read as a real-world accuracy estimate (Chapter 6). The match score is a relative suitability indicator, not a verdict; the system stores no uploaded files and no candidate data, and every shortlist should be reviewed by a person.

CHAPTER 2

CONCEPT EXPLORATION

2.1 Related Approaches

2.1.1 Text representation and similarity

Sparck Jones showed in 1972 that a term is a more useful discriminator when it occurs in fewer documents, and proposed weighting terms by their inverse document frequency [3]. Salton and Buckley later compared many term-weighting schemes experimentally and found that schemes combining a term-frequency component, an inverse-document-frequency component and length normalisation performed well [4]. In the resulting TF-IDF representation each document becomes a sparse vector; after length normalisation, the cosine of the angle between two vectors measures how much weighted vocabulary the documents share [5]. This is exactly what is needed to compare a resume with a job description, and it needs no labelled data, which is why it forms the similarity component of ResumeIQ's score.

2.1.2 Probabilistic text classification

McCallum and Nigam compared two event models for Naive Bayes text classification and found that the multinomial model, which uses word counts, generally performed better than the multi-variate Bernoulli model when the vocabulary was large [6]. Naive Bayes is attractive for a project of this size because it trains in a single pass, works with small amounts of data, gives class probabilities directly and exposes which words drive a decision; it was therefore chosen as the first classifier. Its main weakness is the independence assumption, which makes its probabilities over-confident.

2.1.3 Support Vector Machines for text

Cortes and Vapnik introduced the soft-margin support-vector network [7]. Joachims argued that text categorisation suits SVMs because documents are high-dimensional, sparse and mostly linearly separable, and reported in his experiments that SVMs outperformed k-nearest-neighbour, Naive Bayes, Rocchio and decision-tree classifiers [8]. Linear SVMs for large sparse data are efficiently implemented in LIBLINEAR [9], which underlies scikit-learn's LinearSVC. An SVM outputs a distance from the decision boundary rather than a probability; Platt proposed fitting a sigmoid to these scores to obtain calibrated posterior probabilities [10], which ResumeIQ uses so that the SVM and Naive Bayes outputs can be averaged on the same scale.

2.1.4 Algorithmic hiring and fairness

The hiring literature adds a design constraint that the classification literature does not. Raghavan et al. reviewed how vendors of algorithmic hiring tools describe their bias-mitigation practices and found that public information on how these claims are validated is limited [1], and the 2018 withdrawal of an experimental recruiting tool that penalised women's resumes [2] illustrates how a model trained on historical data can reproduce historical bias. ResumeIQ responds in three modest ways: personal identifiers (name, e-mail, phone, links) are removed before scoring; every score is decomposed into visible components; and the interface and documentation state that the output is a decision aid.

2.2 Summary Table

Ref.	Approach / Model	Setting	Reported Result
[3]	Inverse document frequency term weighting	Document-retrieval experiments	Terms occurring in fewer documents are better discriminators; IDF weighting improved retrieval
[4]	Comparison of term-weighting schemes	Several standard test collections	Schemes combining term frequency, IDF and length normalisation performed well
[6]	Naive Bayes event models (multinomial vs. Bernoulli)	Text-classification corpora	Multinomial model generally better with large vocabularies
[7]	Support-vector network (soft-margin SVM)	Pattern-recognition benchmarks	Introduced the soft-margin SVM with strong benchmark results
[8]	SVM for text categorisation	Reuters and Ohsumed collections	SVMs outperformed k-NN, Naive Bayes, Rocchio and C4.5 in the reported experiments
[10]	Sigmoid calibration of SVM outputs	Support-vector classifiers	Posterior probabilities obtained from SVM scores via a fitted sigmoid
[1]	Review of algorithmic-hiring bias claims	Vendor practices (qualitative)	Limited public information on how bias mitigation is validated

Table 2.1 — Summary of Related Approaches

2.3 What This Told Us

The literature pointed the team toward the same starting point: represent text with TF-IDF, and begin with Naive Bayes and a linear SVM because they are the best-understood classifiers for sparse text and need no architecture decisions beyond a few regularisation settings (Iteration 1, Chapter 4). Because the two models rest on different assumptions (generative and independence-based on one side, margin-based on the other), the team decided to keep both and average their calibrated probabilities in Iteration 2, rather than choose one. The skill taxonomy, the five-part match score, the role-affinity measure and the name-blind scoring were the team's own additions, motivated by the explainability and fairness concerns in [1] and [2]: a transparent rule or a visible component was preferred to an opaque number.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

The project ran across the full PBL cycle, from problem framing at the Zeroth Review (21 June 2026) through the final web-application build and testing. The table below summarises milestones and the work completed; the Mentor Remarks column is reserved for the mentor's own comments and sign-off.

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing, scope (Zeroth Review)	Selected resume screening and job matching as the problem; fixed the input formats (PDF, DOC, DOCX, TXT), the twelve target job roles and the objectives; presented scope and aim at the Zeroth Review.	
3–4	Concept exploration, baseline plan	Surveyed TF-IDF, Naive Bayes, SVM and algorithmic-hiring literature (Chapter 2); decided on a text pipeline with two classifiers; designed the skill taxonomy and role catalog.	
5–7	Iteration 1: baseline model	Implemented PDF/DOCX/TXT text extraction and a first synthetic data generator; trained TF-IDF + Naive Bayes + SVM; obtained 100% hold-out accuracy and recognised that the data were too easy.	

Week	Milestone / Task	Work Done	Mentor Remarks
8–10	Iteration 2: refinement	Made the generator noisier (hybrid, sparse and ambiguous resumes); built the rule-based parser (sections, experience, education), PII removal and the five-part scoring; replaced the one-hot role-fit with a role-affinity measure; added role recommendation.	
11–12	Final approach, implementation, report, demo	Built the Flask web app and JSON API with the screening, job-match and insight pages; wrote 34 automated tests; fixed defects found in testing; containerised with Docker, prepared the Render blueprint and CI workflow; compiled the final report.	

Table 3.1 — Weekly PBL Progress Log

3.2 Requirements

ResumeIQ operates on text extracted from uploaded resumes. The classifiers are trained on 2,400 generated resumes; a command-line option (--csv) retrains the same pipeline on any CSV of labelled resumes, so no code change is needed to move from synthetic to real data. All libraries are open source and the pinned versions below were used for every result in this report [11]–[14].

Category	Requirement
Processor / RAM	Any x86-64 processor; 2 GB RAM or more. The running web worker used about 184 MB of resident memory (measured); no GPU is required
Programming language	Python 3.12 (Docker image); Python 3.13 for local development; HTML, CSS and JavaScript (user interface)
Libraries / frameworks	scikit-learn 1.9.1, NumPy 2.5.3, pandas 3.0.5, joblib 1.6.0 (machine learning); Flask 3.1.3, Gunicorn 26.2.0 (web); pypdf 6.17.0, python-docx 1.2.0, olefile 0.47 (resume parsing)
External tool	antiword (text extraction from legacy .doc files, installed in the Docker image)
Development environment	VS Code; Flask development server; pytest 9 for testing; Chromium (Playwright) for browser verification
Version control / CI	Git and GitHub (repository: sadhanashan16/Resumescreening); GitHub Actions workflow

Category	Requirement
Deployment	Docker container served by Unicorn on Render (render.yaml blueprint) [15]

Table 3.2 — Hardware and Software Requirements

3.3 Feasibility

Training both classifiers, including 5-fold cross-validation, takes about 9 seconds on a single CPU core, and the trained model file is under 3 MB. Screening all fourteen sample resumes (four file formats, including text extraction) took a median of 0.19 s and recommending roles for one resume took about 16 ms, so the complete PBL cycle (baseline, refinement and final system) was achievable within the twelve-week timeframe on ordinary student laptop hardware.

The most time-consuming part of the project was not model training but the parts around the model: the skill taxonomy and its aliases, reliable parsing of dates and degrees, a realistic data generator, and a safe, verified web interface.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

The end-to-end pipeline runs: uploaded resume → text extraction → NLP parsing (sections, skills, education, experience, contact details) → normalisation (personal identifiers removed, skills collapsed to canonical tokens) → TF-IDF vectorisation → Naive Bayes and SVM role probabilities → ensemble → five-part scoring against the job description → ranked shortlist or role recommendations delivered through a Flask web application and JSON API. Figure 4.1 shows this as a block diagram; each block is described below it.

Input and extraction: a resume is accepted as PDF, DOC, DOCX or TXT (up to 30 files of 5 MB each per run). The file type is checked by extension and by content signature, and text is extracted with pypdf, python-docx or antiword; a file with no readable text (for example a scanned image) is rejected with an explanatory message.

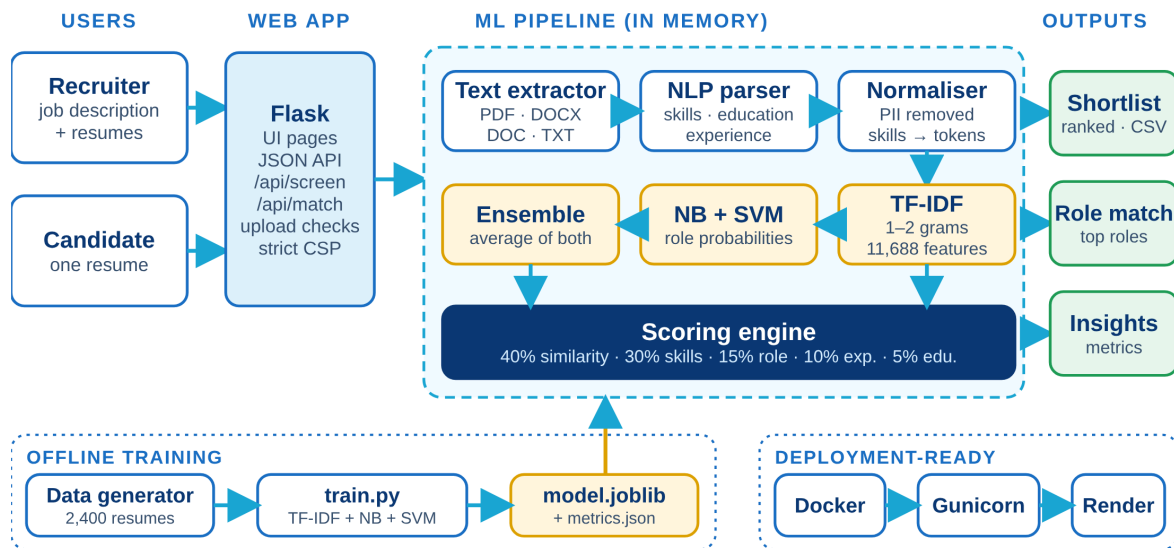


Figure 4.1 — System Architecture Diagram

NLP parsing and normalisation: the parser splits the text into sections, recognises skills from the 164-skill taxonomy, estimates years of experience from date ranges and stated figures, detects the highest degree, and extracts contact details. For scoring, the name, e-mail, phone numbers and links are removed, and every recognised skill is replaced by a single canonical token so that “sklearn” and “scikit-learn” are the same feature.

Classification and scoring: the TF-IDF vectoriser feeds a Multinomial Naive Bayes model and a calibrated linear SVM; their probabilities are averaged into a role distribution. The scoring engine combines this with cosine similarity, skill coverage, experience and education into a 0–100 score. Offline, the training script builds the dataset, fits the models and writes model.joblib and metrics.json, which the web application loads at start-up.

4.2 Iteration 1: Baseline

The simplest working version generated resumes from the role catalog using role-specific experience-bullet templates, vectorised them with TF-IDF (unigrams and bigrams) and trained a Multinomial Naive Bayes classifier and a calibrated linear SVM on an 80/20 stratified split. All three predictors (the two models and their average) scored 1.000 accuracy on the hold-out set and 1.000 in 5-fold cross-validation.

A perfect score on a twelve-class problem was a warning sign rather than a success. Inspection showed that each role's bullet templates were unique to that role, so the label could

be recovered from a handful of template phrases; the model was memorising the generator, not learning to separate roles. This finding motivated Iteration 2.

4.3 Iteration 2: Refinement

Iteration 2 made the data harder and the system more complete. The generator was changed so that resumes could no longer be identified by template phrases:

- 28% of experience bullets are generic (for example “participated in code reviews and daily stand-ups”) and carry no role information; 14% are borrowed from a neighbouring role, and 35% for hybrid resumes.
- 25% of resumes are hybrids that mix the skills of a neighbouring role; 12% use a job title from a neighbouring role; 20% are sparse resumes listing only three to six skills.
- Each candidate lists a random 35–90% of the role's core skills, so some resumes are only weakly characteristic of their role.

With these changes the hold-out accuracy moved to 0.996 (Naive Bayes), 0.992 (SVM) and 0.996 (ensemble), and after the per-candidate skill-rate variation and a larger hybrid share it settled at 0.998 for all three. The data remain highly separable, which is a property of synthetic data, and this is stated as a limitation in Chapter 6 rather than tuned away. As a more meaningful check, five resumes written by hand in a different style from the generator were added to the test suite; all five were classified to the correct role.

Iteration 2 also improved the scoring. The first version measured role fit as the cosine similarity between the resume's and the job's role-probability vectors. Because the classifiers are confident, these vectors are almost one-hot, so a resume for a closely related role scored essentially zero: for the Data Scientist example job, the Machine Learning Engineer sample scored 0.004. Replacing the cosine with a role-affinity measure (full credit for the same role and 0.4 for neighbouring roles) raised it to 0.395 while keeping the Data Scientist sample at 0.983. The grade thresholds were recalibrated from 75/55/35 to 70/50/30 after observing that clearly well-matched resumes scored between the mid-sixties and the low seventies, and name-blind scoring was verified by a test showing that changing a resume's name and e-mail leaves its score unchanged.

4.4 Final Approach

The final system is a two-classifier text pipeline with a transparent scoring layer. Its components are:

- TF-IDF features: unigrams and bigrams, minimum document frequency 2, maximum 0.9, English stop-words removed, sublinear term frequency, L2 normalisation, 11,688 features after fitting. With smoothed inverse document frequency, the weight of term t in document d is shown below, and each vector is then length-normalised so that the dot product of two vectors equals their cosine similarity [3]–[5].
- Multinomial Naive Bayes (smoothing $\alpha = 0.05$) [6]: the class posterior is proportional to the class prior times the product of the word likelihoods raised to their counts.
- Linear SVM (LinearSVC, $C = 0.8$) [7]–[9], calibrated with a sigmoid ($cv = 3$) [10] so that it outputs probabilities.
- Ensemble: the average of the Naive Bayes and calibrated SVM probabilities gives the role distribution of a document.

$$w_{t,d} = (1 + \ln t f_{t,d}) \times (\ln[(1 + N) / (1 + df_t)] + 1)$$

$$P(c | d) \propto P(c) \prod_t P(t | c)^{t f_{t,d}} \quad \cos(a, b) = a \cdot b / (\|a\| \|b\|)$$

The match score of a resume against a job description is a weighted sum of five components, each on a 0–1 scale. When the job description states no experience or education requirement, that component is dropped and the remaining weights are re-normalised (Table 4.1).

$$\text{Score} = 100 \times \sum_k w_k c_k / \sum_k w_k \quad (\text{sum over the components available for the job})$$

Component	Weight	How it is computed
Text similarity	40%	TF-IDF cosine similarity between the name-blind resume and the job description, divided by a ceiling of 0.45 and capped at 1
Skill coverage	30%	Share of the job's technical skills found in the resume; a related skill from the same family (for example MySQL for PostgreSQL) earns half credit
Role fit	15%	$\text{Presume}^T A p_{\text{job}}$, where p are role distributions and A is the role-affinity matrix (1 for the same role, 0.4 for neighbours)
Experience	10%	$\min(1, \text{estimated years} / \text{years required})$; years are the union of dated ranges in the experience section or the stated figure, whichever is larger

Component	Weight	How it is computed
Education	5%	1 if the highest degree meets the requirement, otherwise degree level / required level (1 = school ... 5 = doctorate)

Table 4.1 — Components of the Match Score

Scores are graded Strong (70 or above), Good (50–69), Partial (30–49) and Weak (below 30), and the shortlist is the top N resumes by score. Besides ranking, the engine recommends roles for a single resume by combining text similarity to each role profile (30%), weighted skill fit to the role's core and secondary skills (35%) and the classifier probability (35%), and lists the core skills the resume lacks as skills to learn.

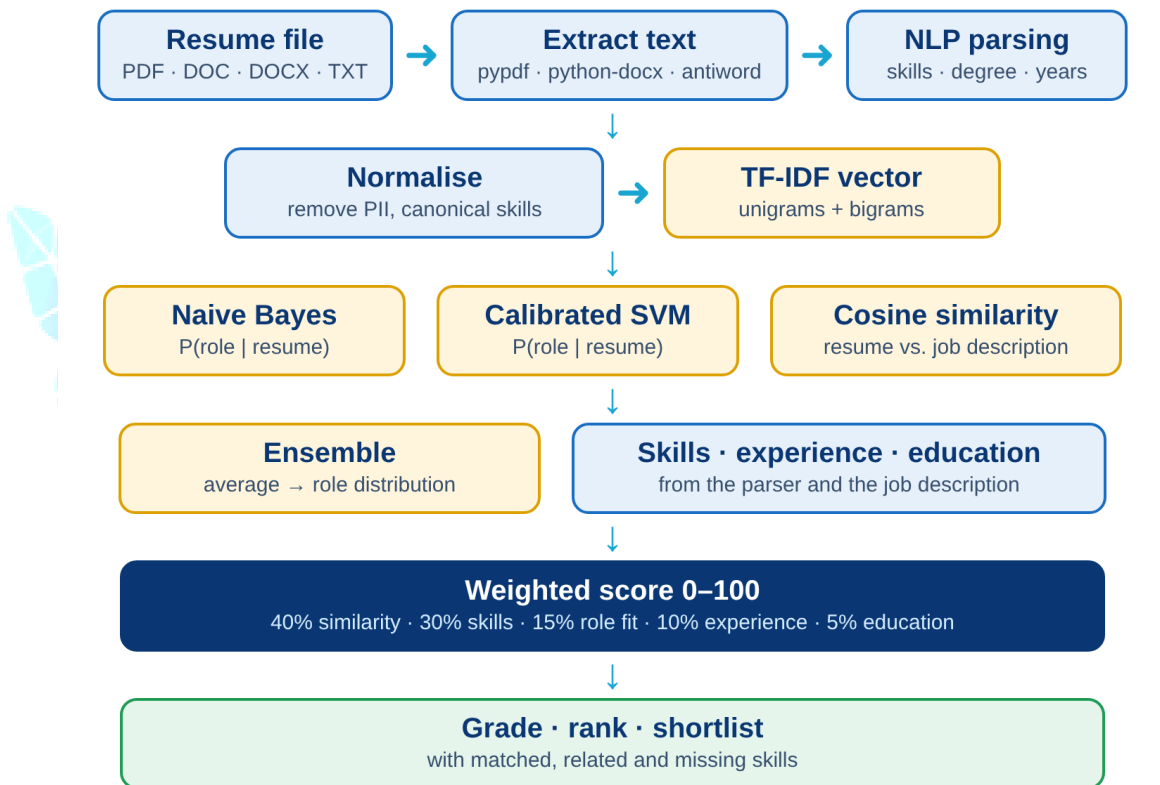


Figure 4.2 — ML Scoring Pipeline Data Flow

Job roles modelled

Job role	Core skills used by the generator and the role profile
Data Scientist	Python, Machine Learning, Statistics, Scikit-learn, Pandas, SQL, Feature Engineering
Machine Learning Engineer	Python, Machine Learning, Deep Learning, TensorFlow, PyTorch, MLOps, Docker

Job role	Core skills used by the generator and the role profile
Data Analyst	SQL, Excel, Data Analysis, Data Visualization, Tableau, Power BI, Business Intelligence
Data Engineer	Python, SQL, ETL, Apache Spark, Airflow, Data Warehousing, Kafka
Backend Developer	REST APIs, SQL, Microservices, Git, Database Design, Data Structures & Algorithms, OOP
Frontend Developer	JavaScript, HTML, CSS, React, Responsive Design, TypeScript, Git
Full Stack Developer	JavaScript, React, Node.js, REST APIs, SQL, HTML, CSS, Git
DevOps Engineer	Docker, Kubernetes, CI/CD, Terraform, AWS, Linux, Jenkins
Mobile App Developer	Android, Kotlin, Mobile Development, Firebase, REST APIs, Git, Java
QA Engineer	Manual Testing, Test Automation, Selenium, Jira, Postman, Performance Testing, Agile
Cybersecurity Analyst	Cybersecurity, SIEM, Incident Response, Network Security, Penetration Testing, Compliance, Linux
Business Analyst	Requirements Gathering, Stakeholder Management, Process Modeling, Gap Analysis, Agile, Documentation, Jira

Table 4.2 — Job Roles Modelled and Their Core Skills

4.5 Training Procedure

The labelled data are split 80/20 with stratification (1,920 training and 480 test resumes, 40 test resumes per role). Five-fold stratified cross-validation is run on the training portion with the TF-IDF vectoriser re-fitted inside every fold, so no vocabulary or document-frequency information leaks from the validation fold. The final vectoriser and classifiers are then fitted on the training split and evaluated once on the untouched test split; the model shipped with the application is exactly the model that the reported hold-out metrics describe.

Component	Configuration
Dataset	2,400 synthetic resumes, 12 roles $\times$ 200, generator seed 42
Split	80/20 stratified: 1,920 training / 480 test; 5-fold stratified cross-validation on the training split
Text normalisation	Contact details removed; skills replaced by canonical tokens; lower-cased; non-alphanumeric characters removed
TF-IDF	ngram (1, 2), min_df 2, max_df 0.9, sublinear tf, English stop-words, max 30,000 features (11,688 used)

Component	Configuration
Naive Bayes	MultinomialNB, alpha = 0.05
SVM	LinearSVC (C = 0.8, random_state 42) wrapped in CalibratedClassifierCV (sigmoid, cv = 3)
Ensemble	Mean of the two predicted probability vectors; arg-max gives the predicted role
Output	model.joblib (vectoriser, both classifiers, labels) and metrics.json

Table 4.3 — Dataset and Model Configuration

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

Machine-learning package: resume_screening/

- extractor.py: converts PDF (pypdf), DOCX (python-docx), DOC (antiword, with an olefile fallback) and TXT files to text, checks file signatures and raises user-readable errors for empty, encrypted or scanned files.
- skills.py: the 164-skill taxonomy with aliases, a case-sensitive guard for ambiguous words (Excel, React, Spring), a regular-expression extractor, and related-skill families used for partial credit.
- parser.py: section detection, contact-detail extraction, experience estimation (union of dated ranges), education-level detection, and job-description parsing.
- text_utils.py: removal of e-mails, URLs and phone numbers, and canonicalisation of skills into vocabulary tokens.
- catalog.py and dataset.py: the twelve job-role profiles and the synthetic resume generator with the noise sources described in Chapter 4.
- train.py: builds the dataset, runs cross-validation, trains Naive Bayes and the calibrated SVM, evaluates on the hold-out set and writes the model and metrics.
- engine.py: loads the model, ranks resumes against a job description, computes the five-part score and recommends roles for a single resume.

Web application

- app.py: the Flask application with four pages (home, screen candidates, find job match, model insights), a JSON API (/api/screen, /api/match, /api/roles, /api/samples, /api/model, /health), upload validation and security headers.
- templates/ and static/: the user interface in HTML, CSS and JavaScript; dynamic content is built with textContent so that uploaded text can never inject markup, and a strict Content Security Policy is enforced.
- tests/: 34 pytest tests; samples/: 14 fictional sample resumes in PDF, DOCX, DOC and TXT formats; Dockerfile, render.yaml and .github/workflows/ci.yml for deployment and CI.

5.2 Key Code Snippets

The normalisation function removes personal identifiers and turns each skill spelling variant into one token, so “sklearn” and “scikit-learn” become the same feature:

```
def normalize_for_model(text: str) -> str:
    """Text fed to the TF-IDF vectoriser.

    Contact details are removed, skills are collapsed to canonical tokens
    (`scikit-learn` / `sklearn` -> `skill_scikit_learn`) so that spelling
    variants share one feature, and everything is lower-cased.
    """
    text = strip_contact_info(text)
    text = canonicalise_skills(text)
    text = text.lower()
    text = _TOKEN_CLEAN.sub(" ", text)
    return re.sub(r"\s+", " ", text).strip()
```

Code 5.1 — Name-blind text normalisation (text_utils.py)

The training script defines the three model components; the SVM is wrapped in a sigmoid calibrator:

```
def make_vectorizer() -> TfidfVectorizer:
    return TfidfVectorizer(
        ngram_range=(1, 2), min_df=2, max_df=0.9, sublinear_tf=True,
        stop_words="english", token_pattern=TOKEN_PATTERN, max_features=30000,
    )

def make_nb() -> MultinomialNB:
    return MultinomialNB(alpha=0.05)

def make_svm() -> CalibratedClassifierCV:
    return CalibratedClassifierCV(LinearSVC(C=0.8, random_state=RANDOM_STATE), cv=3,
        method="sigmoid")
```

Code 5.2 — TF-IDF, Naive Bayes and calibrated SVM definitions (train.py)

Skill coverage gives full credit for a skill the resume lists and half credit for a related skill from the same family:

```
@staticmethod
def _skill_overlap(required: list[str], have: list[str]) -> tuple[float | None, list, list, list]:
    """Coverage of ``required`` by ``have``; related skills earn half credit."""
    technical = [s for s in required if SKILL_BY_NAME[s].category != "soft"]
    req = technical or required
    if not req:
        return None, [], [], []
    have_set = set(have)
    matched, partial, missing, credit = [], [], [], 0.0
    for s in req:
        if s in have_set:
            matched.append(s)
            credit += 1
        else:
            near = sorted(related_skills(s) & have_set)
            if near:
                partial.append({"required": s, "has": near[0]})
                credit += 0.5
            else:
                missing.append(s)
    return credit / len(req), matched, partial, missing
```

Code 5.3 — Skill coverage with related-skill partial credit (engine.py)

The final score is the re-normalised weighted sum of the components that are available for the job description:

```
def score_resume(self, profile: dict, job: dict, sim: float, resume_proba: np.ndarray) -> dict:
    w = dict(config.SCORE_WEIGHTS)
    comps: dict[str, float | None] = {}
    comps["text_similarity"] = min(1.0, max(0.0, sim) / config.SIMILARITY_CEILING)

    cov, matched, partial, missing = self._skill_overlap(job["skills"], profile["skills"])
    comps["skills"] = cov

    comps["role_fit"] = float(resume_proba @ self._affinity @ job["role_probabilities"])

    req_years = job["min_years"]
    comps["experience"] = None if req_years is None else min(1.0, profile["experience_years"] / req_years)

    req_edu = job["education_level"]
    if req_edu is None:
        comps["education"] = None
    else:
        have = profile["education"]["level"]
        comps["education"] = 1.0 if have >= req_edu else (have / req_edu if have else 0.25)

    active = {k: v for k, v in comps.items() if v is not None}
    total_w = sum(w[k] for k in active)
    score = 100 * sum(w[k] * v for k, v in active.items()) / total_w
```

Code 5.4 — Five-part weighted scoring (engine.py, excerpt)

5.3 User Interface / Demo

The home page explains the five-stage pipeline and links to the two tools. In recruiter mode (Screen candidates) the user pastes a job description or loads one of the twelve example descriptions, adds resumes by drag-and-drop (or ticks the built-in sample resumes for a quick demonstration) and chooses the shortlist size.

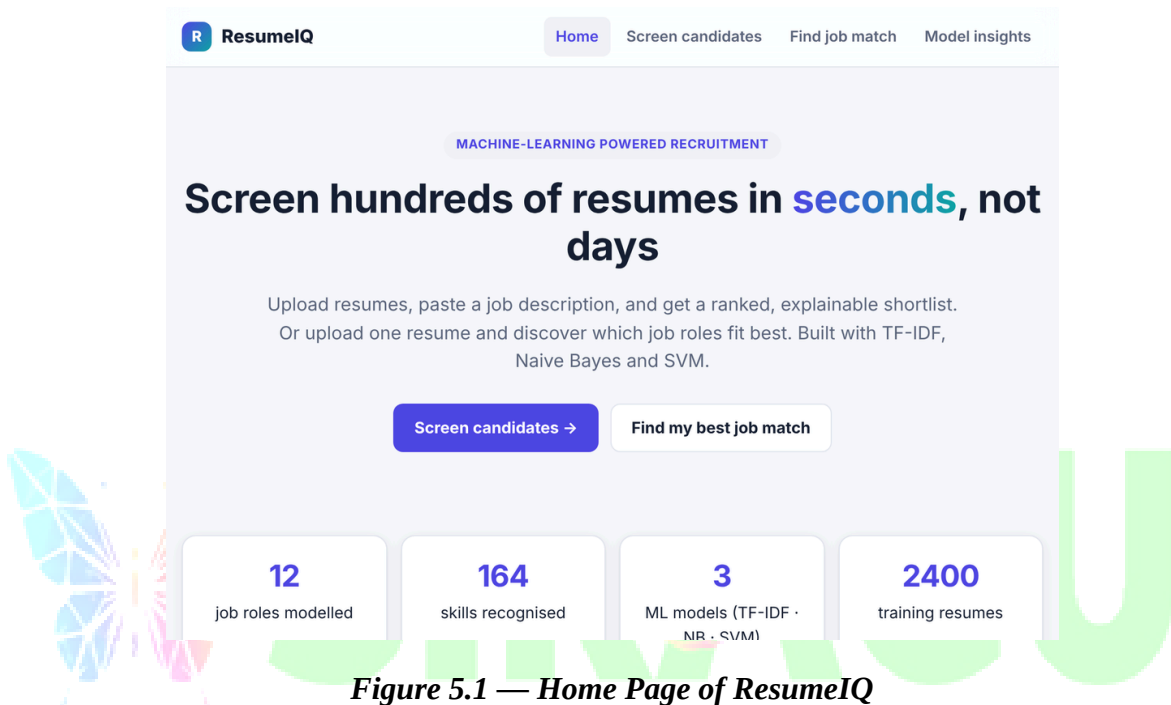


Figure 5.1 — Home Page of ResumeIQ

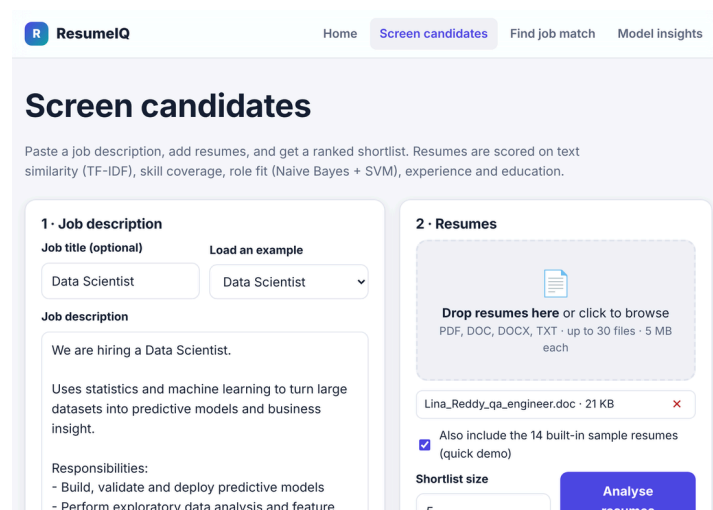


Figure 5.2 — Screen Candidates Page with a Job Description and Resumes

The results page shows the job type detected from the description (with the classifier's confidence), the skills extracted from it and summary counts, followed by one card per candidate in rank order. Each card shows the score and grade, the predicted role, estimated

experience and education, and the matched, related (partial credit) and missing skills; an expandable panel gives the per-component score bars and the separate Naive Bayes and SVM predictions. The shortlist and the full ranking can be downloaded as CSV, with spreadsheet formula characters neutralised.

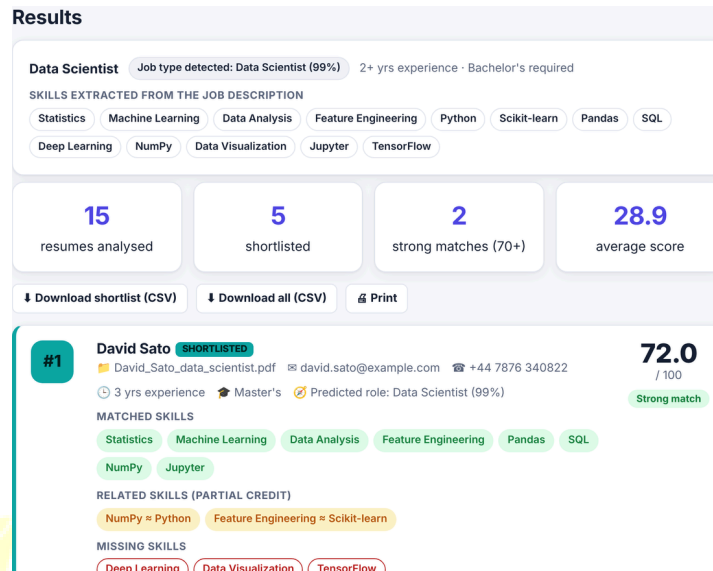


Figure 5.3 — Screening Results: Ranked Candidates with Skill Evidence

In candidate mode (Find job match), a single resume, pasted text or a sample is analysed and the best-fit roles are listed with their scores, the skills already matched and the skills to learn. The Model insights page reports the dataset, the model comparison, the per-role results, the confusion matrix and the most predictive terms, and states prominently that the data are synthetic.

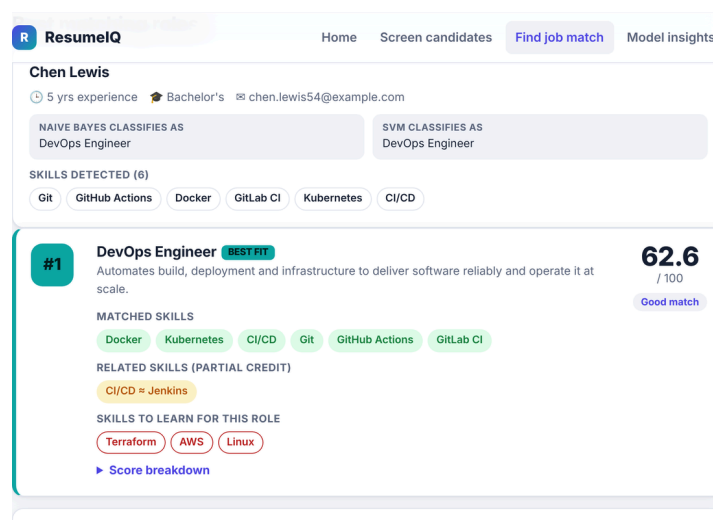


Figure 5.4 — Find Job Match: Role Recommendations for One Resume

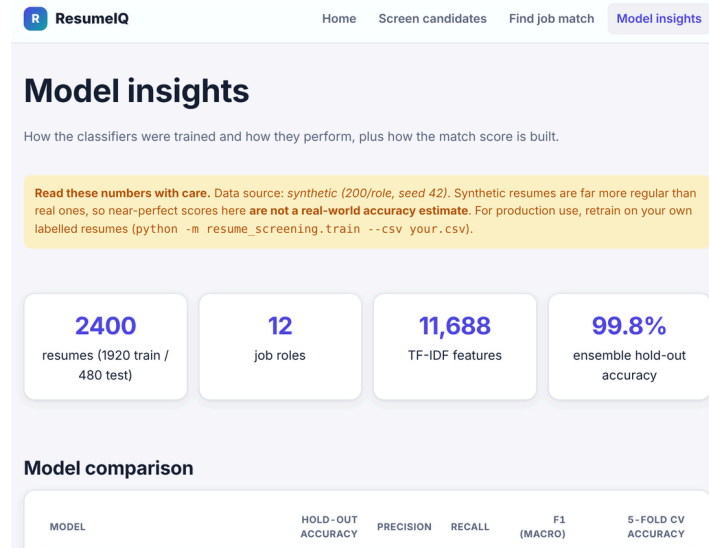


Figure 5.5 — Model Insights Page

Uploaded files are read into memory, scored and discarded; nothing is written to disk or stored. File type is validated by extension and content signature, size and count are limited, error responses never expose internals, and the browser is restricted by a Content Security Policy that blocks inline scripts and styles.

5.4 Interfaces and Deployment

The same engine serves both the web pages and a JSON API, so the system can be used from a browser or called by another program. Table 5.1 lists the endpoints. Uploads are accepted as multipart form data; the resumes are read into memory, scored and discarded, and error responses are returned as JSON with a plain-language message.

Endpoint	Method	Purpose
/	GET	Home page with the pipeline overview
/screen, /match, /insights	GET	Recruiter screening page, single-resume job-match page and model-insights page
/api/screen	POST	Rank uploaded resumes (up to 30) against a job description and return the shortlist as JSON
/api/match	POST	Recommend job roles for one resume (uploaded file, pasted text or a built-in sample)
/api/roles, /api/samples, /api/model	GET	Example job descriptions, sample resume list and the stored training metrics
/health	GET	Health check used by the hosting platform

Table 5.1 — Web Pages and JSON API Endpoints

Deployment follows the standard container route for a machine-learning web service. The Dockerfile starts from a slim Python 3.12 image, installs antiword for legacy .doc files, installs the pinned requirements, and trains the model while the image is built, so the container starts immediately and the deployed model is exactly the one that was evaluated. The application runs under Gunicorn with one worker and four threads, which keeps the resident memory at about 184 MB (measured) and therefore inside the 512 MB limit of Render's free plan. A render.yaml blueprint describes the web service and its health check, and a GitHub Actions workflow runs the tests and builds the image on every push.

Test file	Tests	What it checks
test_skills_parser.py	12	Skill aliases and ambiguous words, Java versus JavaScript, text normalisation, contact details, experience from overlapping dates, education levels, job-description parsing
test_extractor.py	9	TXT, DOCX, PDF and legacy DOC extraction, and friendly errors for empty, corrupt, scanned or unsupported files
test_engine.py	6	Metrics file consistency, five hand-written resumes, ranking order, name-blind scoring, re-weighting when a requirement is missing, role recommendations
test_app.py	7	Page rendering and security headers, screening with uploads, validation of bad input, the 30-file limit, role matching and restricted sample downloads

Table 5.2 — Automated Test Suite (34 tests)

Besides the automated tests, each page was driven in a real browser (Chromium): a job description was loaded, resumes were uploaded, the CSV export was downloaded and the model-insights page was opened, with the browser console checked for script and Content-Security-Policy errors. The deployed configuration was also checked by installing only the pinned requirements into a clean environment, training the model and serving the application with Gunicorn.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Role classification is a supervised problem, so it is evaluated with the standard classification metrics on the 480 held-out resumes: accuracy (share of resumes assigned the correct role), precision and recall (macro-averaged over the twelve roles) and the F1-score, the harmonic

mean of precision and recall. Five-fold cross-validation on the training split gives a second, independent estimate of accuracy. Ranking quality cannot be measured against ground truth because no human ranking of the resumes exists, so it was examined qualitatively: whether resumes of the job's role rise to the top, and whether scores respond sensibly to missing skills, experience and education.

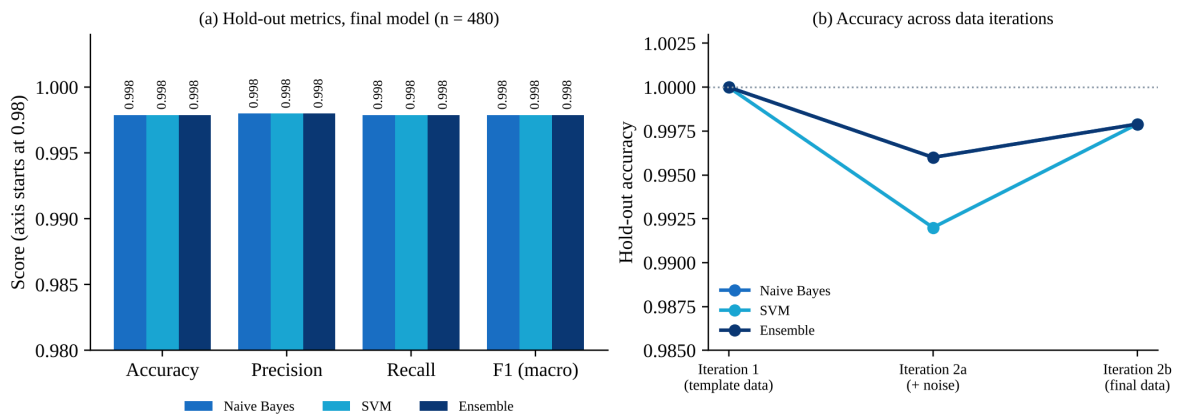


Figure 6.1 — Evaluation Metrics: Final Model and Accuracy Across Data Iterations

Three further instruments were used: (a) a set of five hand-written resumes, in a style different from the generator, that must be classified to the correct role; (b) behavioural tests of the scoring engine (ranking order, name-blindness, re-weighting when a requirement is missing); and (c) 34 automated tests covering the parser, extractor, engine and web interface, plus verification of the pages in a real browser at desktop width and of the screening page at phone width.

6.2 Results Across Iterations

Version	Training data	Hold-out accuracy (NB / SVM / Ensemble)	Key finding
Iteration 1	Template resumes only	1.000 / 1.000 / 1.000	Perfect score; label recoverable from template phrases, so the data were too easy
Iteration 2a	+ generic and borrowed bullets, hybrids, sparse resumes	0.996 / 0.992 / 0.996	Realistic noise lowered accuracy slightly; 5-fold CV 0.997 / 0.996
Final (2b)	+ per-candidate skill-rate variation, 25% hybrids	0.998 / 0.998 / 0.998	5-fold CV 0.997 / 0.994; hand-written resumes 5/5

Version	Training data	Hold-out accuracy (NB / SVM / Ensemble)	Key finding
			correct

Table 6.1 — Model Evaluation Results Across Iterations

On the final model, the Naive Bayes classifier, the SVM and their ensemble each reached 99.8% accuracy (479 of 480 test resumes), with macro precision, recall and F1 of 0.998. The only error is one Cybersecurity Analyst resume classified as a DevOps Engineer, a neighbouring role in the catalog that shares Linux, monitoring and networking skills (Figure 6.2).

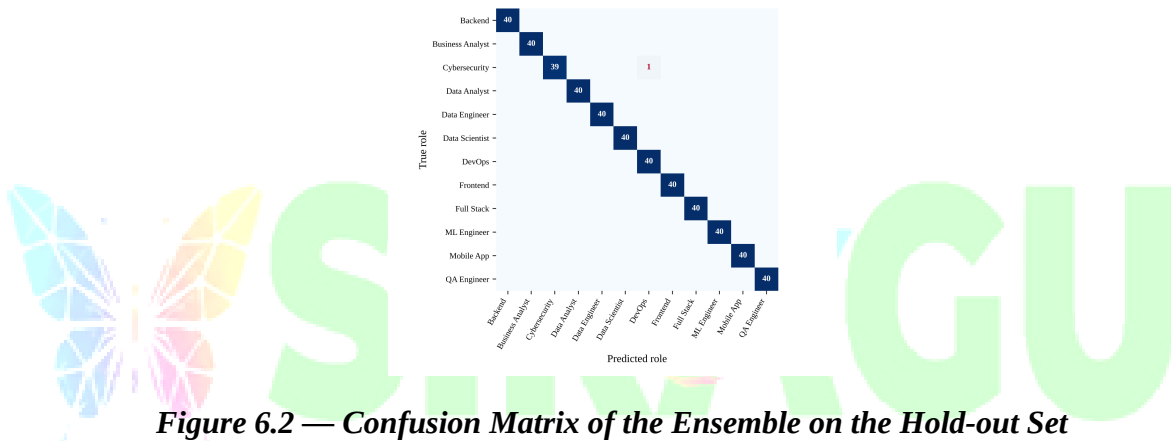


Figure 6.2 — Confusion Matrix of the Ensemble on the Hold-out Set

Job role	Precision	Recall	F1-score	Test resumes
Backend Developer	1.000	1.000	1.000	40
Business Analyst	1.000	1.000	1.000	40
Cybersecurity Analyst	1.000	0.975	0.987	40
Data Analyst	1.000	1.000	1.000	40
Data Engineer	1.000	1.000	1.000	40
Data Scientist	1.000	1.000	1.000	40
DevOps Engineer	0.976	1.000	0.988	40
Frontend Developer	1.000	1.000	1.000	40
Full Stack Developer	1.000	1.000	1.000	40
Machine Learning Engineer	1.000	1.000	1.000	40
Mobile App Developer	1.000	1.000	1.000	40
QA Engineer	1.000	1.000	1.000	40

Table 6.2 — Per-role Results of the Ensemble on the Hold-out Set

To see the ranking behave end to end, the fourteen built-in sample resumes were screened against the example Data Scientist job description (a median of 0.19 s for all fourteen, including text extraction from PDF, DOCX, DOC and TXT files). Figure 6.3 and Table 6.3 show the highest-ranked candidates.

Rank	Candidate	Predicted role	Score	Grade	Text sim.	Skills
1	David Sato	Data Scientist	72.0	Strong	54	69
2	James Mensah	Data Scientist	70.7	Strong	48	73
3	Isabella Nair	Data Analyst	46.6	Partial	18	62
4	Sneha Mehta	Machine Learning Engineer	40.7	Partial	9	54
5	Ingrid Rossi	Data Engineer	35.5	Partial	11	54
6	Lina Muller	Full Stack Developer	23.1	Weak	3	23
7	Kwame Patel	Business Analyst	20.8	Weak	3	15
8	Aiko Lewis	Frontend Developer	16.6	Weak	4	0

Table 6.3 — Top Candidates for the Data Scientist Job (14 Sample Resumes)

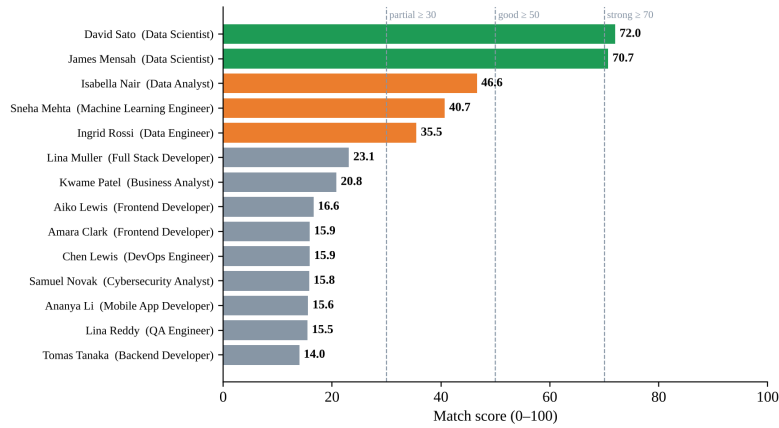


Figure 6.3 — Live Ranking of the Sample Resumes for the Data Scientist Job

6.3 Discussion

The most consistent observation is that the two classifiers agree: Naive Bayes and the calibrated SVM predicted the same role for every resume in the sample screening, and their one-error performance on the hold-out set is shared. This is partly a property of the synthetic data, in which each role has a distinctive skill vocabulary, but it also means that the ensemble

adds robustness rather than changing outcomes here. Its value would be expected to show on noisier real resumes, where a probabilistic and a margin-based model are likely to disagree more often; this was not tested.

The scoring layer, rather than the classifier, is what makes the output useful to a recruiter. In the Data Scientist example the two Data Scientist resumes scored in the strong band (72.0 and 70.7), related roles scored in the partial band because they share part of the skill set, and unrelated roles scored below 30. Because every component is shown, a recruiter can see why a candidate ranked where they did, for instance a missing Deep Learning skill or an experience shortfall, and can override the score. The related-skill rule (half credit) and the role-affinity measure were both introduced because the first, stricter versions penalised candidates for what a human reader would call near-misses.

Name-blind scoring was verified directly: replacing a resume's name and e-mail address leaves its score unchanged. This removes one channel through which bias could enter, but it is not a fairness guarantee: other fields, such as institution names or employment gaps, can still correlate with protected characteristics in real data, and a proper fairness audit would need real, demographically annotated resumes that were not available to the team.

Several defects were found and fixed only because the system was tested as a real web service rather than as a notebook. A file-name fallback produced “Priya Sharma Resume” instead of “Priya Sharma” because an underscore is not a word boundary in a regular expression; inline style attributes were blocked by the Content Security Policy and silently removed spacing; the spreadsheet-injection guard in the CSV export altered phone numbers that begin with “+”; and the confusion matrix headers were clipped on the insights page. Each was corrected and covered by a test or a browser check.

6.4 Limitations

- The classifiers were trained on 2,400 synthetic resumes. The 99.8% hold-out accuracy shows that the pipeline works and that the roles are separable in the generated data; it is not a real-world accuracy estimate, and real resumes are expected to give lower and more variable results. The test suite contains five hand-written resumes as a modest sanity check, not as a substitute for real evaluation.
- Skill, education and experience extraction is rule-based over a 164-skill taxonomy; skills, phrasings and date formats outside these rules are missed, and a trained named-

entity model would be more robust.

- Scanned or image-only resumes are rejected because there is no optical character recognition; very unusual layouts (multi-column designs, text in images) can reduce extraction quality.
- The score weights, the similarity ceiling (0.45), the affinity value (0.4) and the grade thresholds are design choices made by the team and calibrated on the sample resumes, not parameters learned from recruiter decisions.
- The system supports twelve roles and English text only, keeps no database or user accounts, and is a decision aid that requires human review.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

Sadhana Shanmugam: I focused on the machine-learning pipeline: the text extractor, the skill taxonomy and parser, the TF-IDF, Naive Bayes and SVM training code, the data generator and the scoring engine. The biggest thing I learned is that a perfect score is a question, not an answer: when the first model reached 100% accuracy I had to go back and discover that the generator, not the model, was doing the work. Making the data realistically hard, and then accepting that it was still easy and saying so, taught me more about evaluation than the models themselves. The hardest part was the parser: dates, degrees and skills are written in so many ways that every rule had to be tested against awkward examples.

Dhanya Sri Mohandass: I focused on the web interface, the sample resumes, the automated tests and the deployment files. Building the interface in plain HTML, CSS and JavaScript with a strict Content Security Policy showed me how many small things can go wrong between a working model and a usable product: blocked inline styles, clipped chart labels, and a CSV export that quietly changed phone numbers. The most valuable habit was checking the pages in a real browser, including at phone width, instead of trusting that the code looked correct, and writing a test for each defect once it had been found.

7.2 Team Learning

Splitting ownership along the pipeline/interface boundary worked well once the JSON response format of the engine was fixed early, because it allowed both members to work in

parallel: the interface was built against a documented response shape while the scoring logic was still being refined underneath it. What we would do differently if we restarted the cycle is lock that response format even earlier; several interface components had to be reworked when the score breakdown and the related-skills field were added in Iteration 2.

The change that most improved the project was the decision, after the first perfect score, to treat the evaluation itself as something to be tested. It led to a harder data generator, to a hand-written sanity check, to an honest limitations section, and to scoring components that a recruiter can read instead of a single opaque number. Both members also learned to review each other's work before every milestone, which caught several defects before they reached the demonstration.

7.3 Course Outcomes — Evidence Summary



Figure 7.1 — Course Outcomes

- **Technical / ML competency:** the Naive Bayes and SVM text classifiers, TF-IDF features, probability calibration and cosine-similarity scoring, evaluated with a hold-out set and cross-validation, demonstrate applied understanding of supervised text classification (Chapters 2 and 4).
- **Problem framing and iteration:** the weekly progress log (Table 3.1) and the Iteration 1 → Iteration 2 → Final Approach progression (Chapter 4) show the build evolving in response to specific findings, notably the first perfect score and the one-hot role-fit result.
- **Teamwork and division of labour:** the Team Roles table and the reflections above (Section 7.1) show a consistent pipeline/interface split with equal contribution and

joint review before each milestone.

- Engineering rigour beyond the model: 34 automated tests, browser verification, defects found and fixed (Chapter 6), upload validation, a strict Content Security Policy and a Docker-based deployment show that the system was treated as a service to be verified, not only a model to be fitted.
- Honest self-assessment: Chapter 6 states plainly that the data are synthetic and that the headline accuracy is not a real-world estimate, which is the candid, reflective evaluation the PBL format asks for.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

ResumeIQ demonstrates that a classical machine-learning pipeline (TF-IDF features, a Multinomial Naive Bayes model and a calibrated linear SVM) combined with rule-based NLP extraction and a transparent five-part score can turn unstructured resumes into a ranked, explainable shortlist and into role recommendations, without using a candidate's name or contact details and on ordinary laptop hardware. On 480 held-out synthetic resumes the ensemble reached 99.8% accuracy, all five hand-written resumes were matched to the correct role, and fourteen sample resumes in four file formats were screened in about 0.19 s. Delivered as a tested Flask web application with a JSON API and a Docker and Render deployment package, the project addressed its driving question directly: text-based models can separate job roles and rank resumes in a way that a recruiter can inspect. Because the training data are synthetic, the numerical results show that the pipeline works rather than how accurate it would be on real resumes.

8.2 Future Scope

- Train and validate on real, labelled resumes (the --csv option already retrains the same pipeline), with the consent and anonymisation procedures that personal data requires, and report real-world accuracy.
- Replace the rule-based extractor with a trained named-entity model and extend the skill taxonomy and role catalog beyond twelve roles and English-language resumes.
- Add optical character recognition so that scanned resumes can be processed.

- Carry out a fairness audit on real, demographically annotated data, and add monitoring so that score distributions can be compared across groups.
- Learn the score weights and the similarity ceiling from recruiter decisions instead of fixing them by hand, and add recruiter feedback to refine rankings.
- Add persistent storage, user accounts and audit logging, and deploy the packaged container on Render with usage monitoring.

REFERENCES

- [1] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in Proc. ACM Conf. on Fairness, Accountability, and Transparency (FAT*), Barcelona, Spain, 2020, pp. 469–481.
- [2] J. Destin, "Amazon scraps secret AI recruiting tool that showed bias against women," Reuters, Oct. 10, 2018.
- [3] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.
- [4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.
- [5] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.
- [6] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in Proc. AAAI-98 Workshop on Learning for Text Categorization, 1998, pp. 41–48.
- [7] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [8] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. European Conf. on Machine Learning (ECML), 1998, pp. 137–142.
- [9] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," Journal of Machine Learning Research, vol. 9, pp. 1871–1874, 2008.
- [10] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, Cambridge, MA, USA: MIT Press, 1999, pp. 61–74.
- [11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [12] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.
- [13] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 51–56.
- [14] Pallets Projects, "Flask documentation." [Online]. Available: <https://flask.palletsprojects.com/> (accessed 2026).
- [15] Render, "Blueprint specification (render.yaml)." [Online]. Available: <https://render.com/docs/blueprint-spec> (accessed 2026).

APPENDIX

A.1 Full Source Code

[<https://github.com/sadhanashan16/Resumescreeing>]

A.2 Complete Weekly PBL Log

The full week-by-week log is summarised in Table 3.1 (Chapter 3); this build's log spans the twelve-week PBL cycle beginning at the Zeroth Review (21 June 2026). The Mentor Remarks column of Table 3.1 is left for the mentor to complete and sign.

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
Sadhana Shanmugam	50%	50%	Built the extraction, NLP parsing, model training and scoring engine.
Dhanya Sri Mohandass	50%	50%	Built the web interface, tests, sample data and deployment package.